

Реализация основных моделей в агентном подходе

Дисциплина: Имитационное моделирование

Верниковская Екатерина Андреевна

Содержание

1	Реализация основных моделей в агентном подходе	5
1.1	Модель SIR	5
1.2	Агентный подходе	5
1.3	Эпидемиологическая модель SIR в агентном подходе	6
1.4	Код модели (src/sir_model.jl)	6
1.5	Базовый эксперимент	11
1.6	Сканирование коэффициента заразности	17
1.7	Исследование эффекта миграции	25
1.8	Сводная визуализация результатов	31

Список иллюстраций

Список таблиц

1 Реализация основных моделей в агентном подходе

1.1 Модель SIR

Модель SIR - это фундаментальная математическая модель в эпидемиологии, которая описывает распространение инфекционного заболевания в закрытой популяции. Название происходит от трёх групп (компарментов), на которые делится популяция:

- S - Susceptible (Восприимчивые): люди, которые не болели, не имеют иммунитета и могут заразиться
- I - Infectious (Инфицированные/Заразные): люди, которые в данный момент больны и могут передавать инфекцию
- R - Recovered (Выздоровевшие/Удаленные): люди, которые переболели и приобрели иммунитет (или умерли). Они больше не участвуют в процессе передачи

1.2 Агентный подходе

Показывает переход от классических дифференциальных моделей (SIR) к агентному моделированию. Здесь эпидемия моделируется не уравнениями, а поведением каждого человека (агента) индивидуально.

1.3 Эпидемиологическая модель SIR в агентном подходе

Основная идея: вместо того чтобы решать систему дифференциальных уравнений:

$$\begin{aligned}\frac{dS}{dt} &= -\frac{\beta IS}{N} \\ \frac{dI}{dt} &= \frac{\beta IS}{N} - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}$$

Мы создаем каждого человека как отдельного агента, который:

- Имеет состояние (S, I, R)
- Может заражать соседей
- Может перемещаться между городами
- Может выздороветь или умереть

1.4 Код модели (src/sir_model.jl)

Определение агента:

- `@agent` — макрос из `Agents.jl`, создающий тип агента
- `Person` — имя нашего агента (человек)
- `GraphAgent` — тип пространства: агенты живут в узлах графа (города)
- `days_infected` — счетчик дней болезни (нужен, чтобы знать, когда выздоравливать)
- `status` — текущее состояние: восприимчивый (:S), зараженный (:I), выздоровевший (:R)

```
@agent struct Person(GraphAgent)
    days_infected::Int # количество дней с момента заражения
    status::Symbol
    # :S, :I, :R
end
```

Инициализация модели:

```
function initialize_sir(;
    Ns = [1000, 1000, 1000], # численность по трём городам
    migration_rates = nothing, # матрица миграции
     $\beta_{und}$  = [0.5, 0.5, 0.5], # передача невыявленными
     $\beta_{det}$  = [0.05, 0.05, 0.05], # передача выявленными
    infection_period = 14,
    detection_time = 7,
    death_rate = 0.02,
    reinfection_probability = 0.1,
    Is = [0, 0, 1], # начальное количество инфицированных
    seed = 42,
    n_steps = 100,
)
```

Важные параметры:

- N_s — в каждом из 3 городов по 1000 человек
- $\beta_{und} = 0.5$ — высокая заразность до выявления (человек не знает, что болен)
- $\beta_{det} = 0.05$ — низкая заразность после выявления (человек изолирован)
- $detection_time = 7$ — на 7-й день болезни человека выявляют (тест, симптомы)

Матрица миграции:

```
if migration_rates == nothing
    migration_rates = zeros(C, C)
    for i = 1:C
        for j = 1:C
            migration_rates[i, j] = (Ns[i] + Ns[j]) / Ns[i]
        end
    end
    for i = 1:C
        migration_rates[i, :] ./= sum(migration_rates[i, :])
    end
end
```

Смысл: Создается матрица вероятностей переезда между городами. Вероятность переехать из города i в город j пропорциональна сумме населения этих городов. Чем больше город назначения, тем выше вероятность туда поехать.

Шаг агента:

```
function sir_agent_step!(agent, model)
    migrate!(agent, model)          # 1. Может переехать
    if agent.status == :I
        transmit!(agent, model)     # 2. Если болен - может заразить
    end
    if agent.status == :I
        agent.days_infected += 1     # 3. Увеличиваем счетчик болезни
    end
    recover_or_die!(agent, model)    # 4. Проверяем, пора ли выздоравливать/умирать
end
```


Порядок действий каждый день:

1. **Миграция** — агент может переехать в другой город
2. **Передача инфекции** — если агент болен, он может заразить других
3. **Обновление счетчика** — увеличиваем количество дней болезни
4. **Выздоровление/смерть** — если прошло `infection_period` дней

Передача инфекции:

```
function transmit!(agent, model)
    # Определяем заразность в зависимости от стадии болезни
    rate = if agent.days_infected < model.detection_time
        model.β_und[agent.pos]      # до выявления - высокая заразность
    else
        model.β_det[agent.pos]      # после выявления - низкая заразность
    end

    # Сколько человек заразит сегодня (пуассоновский процесс)
    n_infections = rand(abmrng(model), Poisson(rate))

    # Получаем всех соседей в том же городе
    neighbors = [a for a in agents_in_position(agent.pos, model) if a.id != agent.id]
    shuffle!(abmrng(model), neighbors)

    for contact in neighbors
        # Заражаем восприимчивых
        if contact.status == :S
            contact.status = :I
            contact.days_infected = 1
            n_infections -= 1
        end
    end
end
```

```

        n_infections == 0 && return nothing
      # Или повторно заражаем выздоровевших с вероятностью reinfection_probab
    elseif contact.status == :R && rand(abmrng(model)) ≤ model.reinfection_
      contact.status = :I
      contact.days_infected = 1
      n_infections -= 1
      n_infections == 0 && return nothing
    end
  end
end
end

```

****Ключевая идея:** Заразность зависит от того, выявлен ли больной. Выявленные больные (после detection_time дней) имеют меньшую заразность, так как они изолированы

Выздоровление или смерть:

```

function recover_or_die!(agent, model)
  if agent.status == :I && agent.days_infected ≥ model.infection_period
    if rand(abmrng(model)) ≤ model.death_rate
      remove_agent!(agent, model) # умирает
    else
      agent.status = :R           # выздоравливает
      agent.days_infected = 0
    end
  end
end
end

```

После infection_period дней (14) происходит одно из двух:

- С вероятностью death_rate (2%) агент умирает (удаляется из модели) -
Иначе выздоравливает и получает иммунитет (статус R)

Базовый эксперимент (scripts/sir_run_basic.jl) - Запускает один эксперимент с параметрами по умолчанию и строит график динамики эпидемии.

Сканирование коэффициента заразности (scripts/sir_scan_beta.jl) - Исследует, как изменение заразности β влияет на эпидемию. Запускает много экспериментов с разными β и усредняет результаты.

Исследование эффекта миграции (scripts/sir_migration_effect.jl) - Исследует, как интенсивность перемещения людей между городами влияет на скорость распространения эпидемии.

Сводная визуализация (scripts/sir_visualize_dynamics.jl) - Загружает результаты параметрического сканирования из файла beta_scan_all.csv и строит составной график из трех панелей.

1.5 Базовый эксперимент

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots
default(fmt = :png)
using JLD2
```

```
include(scrdir("sir_model.jl"))
```

```
Activating project at `~/work/study/2025-2026/Имитационное моделирование/simu
modeling/labs/lab03/project`
```

```
└ Warning: DrWatson could not find find a project file by recursively checking
  | (given dir: /home/eavernikovskaya)
```

L @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84

total_count (generic function with 1 method)

Параметры эксперимента

```
params = Dict(  
    :Ns => [1000, 1000, 1000],  
    :β_und => [0.5, 0.5, 0.5],  
    :β_det => [0.05, 0.05, 0.05],  
    :infection_period => 14,  
    :detection_time => 7,  
    :death_rate => 0.02,  
    :reinfection_probability => 0.1,  
    :Is => [0, 0, 1],  
    :seed => 42,  
    :n_steps => 100,  
)
```

Dict{Symbol, Any} with 10 entries:

:Is	=> [0, 0, 1]
:seed	=> 42
:infection_period	=> 14
:n_steps	=> 100
:β_und	=> [0.5, 0.5, 0.5]
:Ns	=> [1000, 1000, 1000]
:detection_time	=> 7
:reinfection_probability	=> 0.1
:β_det	=> [0.05, 0.05, 0.05]
:death_rate	=> 0.02

Инициализация модели

```
model = initialize_sir(; params...)
```

StandardABM with 3000 agents of type Person

agents container: Dict

space: GraphSpace with 3 positions and 3 edges

scheduler: fastest

properties: C, infection_period, β_{und} , Ns, migration_rates, detection_time, r

Подготовка массивов для хранения данных

```
times = Int[]
```

```
S_vals = Int[]
```

```
I_vals = Int[]
```

```
R_vals = Int[]
```

```
total_vals = Int[]
```

```
Int64[]
```

Запуск симуляции вручную

```
for step = 1:params[:n_steps]
```

```
    Agents.step!(model, 1)
```

```
    push!(times, step)
```

```
    push!(S_vals, susceptible_count(model))
```

```
    push!(I_vals, infected_count(model))
```

```
    push!(R_vals, recovered_count(model))
```

```
    push!(total_vals, total_count(model))
```

```
end
```

Создаём DataFrame для удобства (опционально)

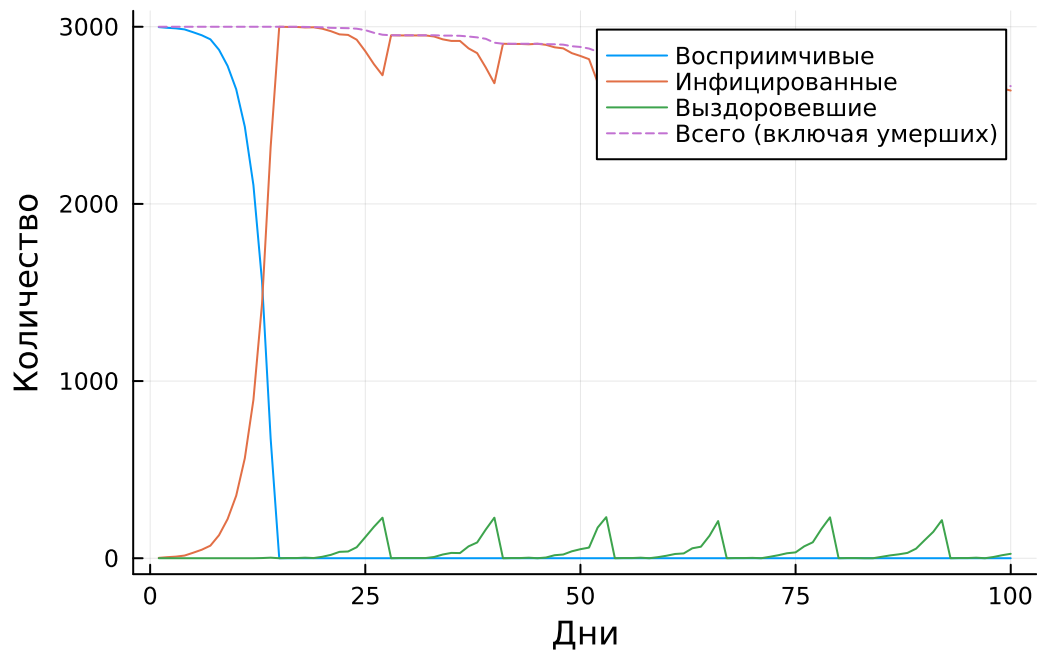
```
agent_df = DataFrame(time = times, susceptible = S_vals, infected  
    ↪ = I_vals, recovered = R_vals)  
model_df = DataFrame(time = times, total = total_vals)
```

	time	total
	Int64	Int64
1	1	3000
2	2	3000
3	3	3000
4	4	3000
5	5	3000
6	6	3000
7	7	3000
8	8	3000
9	9	3000
10	10	3000
11	11	3000
12	12	3000
13	13	3000
14	14	3000
15	15	3000
16	16	3000
17	17	3000
18	18	2999
19	19	2998
20	20	2997
21	21	2994
22	22	2993
23	23	2992
24	24	2989
25	25	2981
26	26	2966
27	27	2955
28	28	2952
29	29	2952
30	30	2952
...	...	...

```
p = plot(
    agent_df.time,
    agent_df.susceptible,
    label = "Восприимчивые",
    xlabel = "Дни",
    ylabel = "Количество",
)

plot!(p, agent_df.time, agent_df.infected, label =
    ↪ "Инфицированные")
plot!(p, agent_df.time, agent_df.recovered, label =
    ↪ "Выздоровевшие")
plot!(p, agent_df.time, model_df.total, label = "Всего (включая
    ↪ умерших)", linestyle = :dash)
savefig(plotsdir("sir_basic_dynamics.png"))

p
```



Сохранение данных

```
@save datadir("sir_basic_agent.jld2") agent_df
@save datadir("sir_basic_model.jld2") model_df
```

1.6 Сканирование коэффициента заразности

```
using DrWatson
```

```
@quickactivate "project"
```

```
using Agents, DataFrames, Plots, CSV, Random
default(fmt = :png)
```

```
include(sourcedir("sir_model.jl"))
```

```
└ Warning: DrWatson could not find find a project file by recursively checking
```

```
| (given dir: /home/eavernikovskaya)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

total_count (generic function with 1 method)

Функция для запуска одного эксперимента и возврата метрик

```
function run_experiment(p)
    beta = p[:beta]
    β_und = fill(beta, 3)
    β_det = fill(beta/10, 3)
    model = initialize_sir(;
        Ns = p[:Ns],
        β_und = β_und,
        β_det = β_det,
        infection_period = p[:infection_period],
        detection_time = p[:detection_time],
        death_rate = p[:death_rate],
        reinfection_probability = p[:reinfection_probability],
        Is = p[:Is],
        seed = p[:seed],
        n_steps = p[:n_steps],
    )

    infected_fraction(model) = count(a.status == :I for a in
        ↪ allagents(model)) / nagents(model)
    peak_infected = 0.0

    for step = 1:p[:n_steps]
        agent_ids = collect(allids(model))
```

```

    for id in agent_ids
        agent = try
            model[id]
        catch
            nothing
        end
        if agent !== nothing
            sir_agent_step!(agent, model)
        end
    end
    frac = infected_fraction(model)
    if frac > peak_infected
        peak_infected = frac
    end
end

final_infected = infected_fraction(model)
final_recovered = count(a.status == :R for a in
    ↪ allagents(model)) / nagents(model)
total_deaths = sum(p[:Ns]) - nagents(model)

return (
    peak = peak_infected,
    final_inf = final_infected,
    final_rec = final_recovered,
    deaths = total_deaths,
)
end

```

run_experiment (generic function with 1 method)

Диапазон значений beta (скаляр)

```
beta_range = 0.1:0.1:1.0  
seeds = [42, 43, 44]
```

3-element Vector{Int64}:

42

43

44

Создаём список параметров

```
params_list = []  
for b in beta_range  
    for s in seeds  
        push!(  
            params_list,  
            Dict(  
                :beta => b,  
                :Ns => [1000, 1000, 1000],  
                :infection_period => 14,  
                :detection_time => 7,  
                :death_rate => 0.02,  
                :reinfection_probability => 0.1,  
                :Is => [0, 0, 1],  
                :seed => s,  
                :n_steps => 100,  
            )  
        )  
    end  
end
```

```
        ),  
    )  
end  
end
```

Запуск экспериментов

```
results = []  
for params in params_list  
    data = run_experiment(params)  
    push!(results, merge(params, Dict{pairs(data)}))  
    println("Завершён эксперимент с beta = $(params[:beta]), seed  
↪      = $(params[:seed])")  
end
```

```
Завершён эксперимент с beta = 0.1, seed = 42  
Завершён эксперимент с beta = 0.1, seed = 43  
Завершён эксперимент с beta = 0.1, seed = 44  
Завершён эксперимент с beta = 0.2, seed = 42  
Завершён эксперимент с beta = 0.2, seed = 43  
Завершён эксперимент с beta = 0.2, seed = 44  
Завершён эксперимент с beta = 0.3, seed = 42  
Завершён эксперимент с beta = 0.3, seed = 43  
Завершён эксперимент с beta = 0.3, seed = 44  
Завершён эксперимент с beta = 0.4, seed = 42  
Завершён эксперимент с beta = 0.4, seed = 43  
Завершён эксперимент с beta = 0.4, seed = 44  
Завершён эксперимент с beta = 0.5, seed = 42  
Завершён эксперимент с beta = 0.5, seed = 43
```

Завершён эксперимент с $\beta = 0.5$, seed = 44
Завершён эксперимент с $\beta = 0.6$, seed = 42
Завершён эксперимент с $\beta = 0.6$, seed = 43
Завершён эксперимент с $\beta = 0.6$, seed = 44
Завершён эксперимент с $\beta = 0.7$, seed = 42
Завершён эксперимент с $\beta = 0.7$, seed = 43
Завершён эксперимент с $\beta = 0.7$, seed = 44
Завершён эксперимент с $\beta = 0.8$, seed = 42
Завершён эксперимент с $\beta = 0.8$, seed = 43
Завершён эксперимент с $\beta = 0.8$, seed = 44
Завершён эксперимент с $\beta = 0.9$, seed = 42
Завершён эксперимент с $\beta = 0.9$, seed = 43
Завершён эксперимент с $\beta = 0.9$, seed = 44
Завершён эксперимент с $\beta = 1.0$, seed = 42
Завершён эксперимент с $\beta = 1.0$, seed = 43
Завершён эксперимент с $\beta = 1.0$, seed = 44

Сохраняем все прогоны

```
df = DataFrame(results)
CSV.write(datadir("beta_scan_all.csv"), df)
```

"/home/eavernikovskaya/work/study/2025-2026/Имитационное моделирование/simulation
modeling/labs/lab03/project/data/beta_scan_all.csv"

Усреднение по повторам

using Statistics

```
grouped = combine(
    groupby(df, [:beta]),
    :peak ⇒ mean ⇒ :mean_peak,
    :final_inf ⇒ mean ⇒ :mean_final_inf,
    :deaths ⇒ mean ⇒ :mean_deaths,
)
```

	beta	mean_peak	mean_final_inf	mean_deaths
	Float64	Float64	Float64	Float64
1	0.1	0.000888889	0.0	0.0
2	0.2	0.332206	0.319967	44.0
3	0.3	0.666659	0.643955	208.0
4	0.4	0.666889	0.629722	226.667
5	0.5	1.0	0.996625	317.667
6	0.6	1.0	0.993352	348.0
7	0.7	1.0	0.962127	361.333
8	0.8	1.0	0.946348	356.333
9	0.9	1.0	0.963478	385.667
10	1.0	1.0	1.0	407.333

График

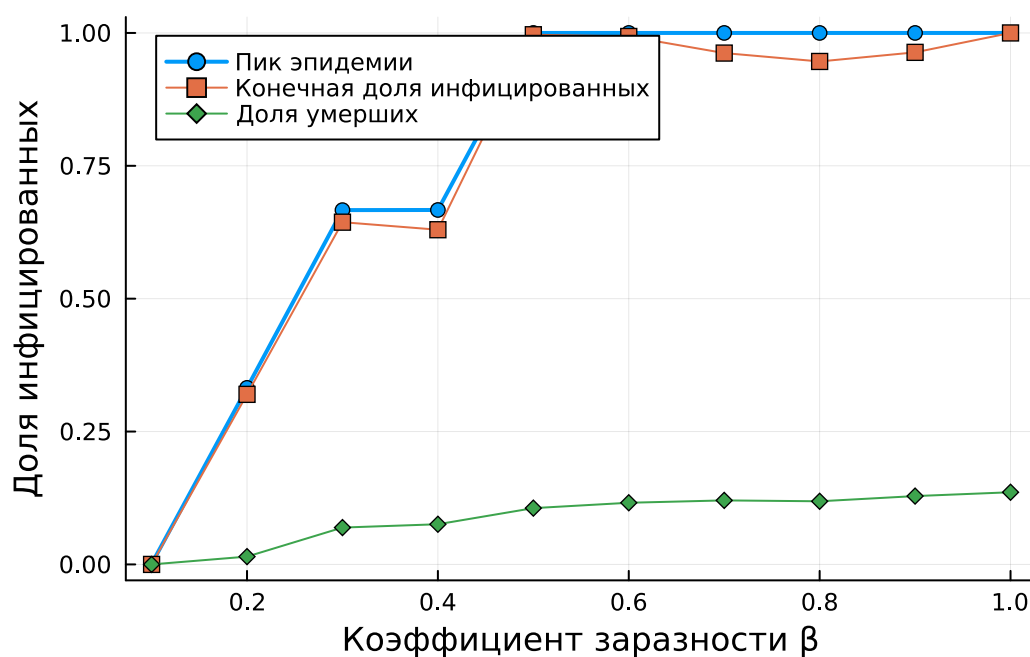
```
p1 = plot(
    grouped.beta,
    grouped.mean_peak,
    label = "Пик эпидемии",
    xlabel = "Коэффициент заразности  $\beta$ ",
    ylabel = "Доля инфицированных",
    marker = :circle,
    linewidth = 2,
```

)

```
plot!(p1, grouped.beta, grouped.mean_final_inf, label = "Конечная  
↪ доля инфицированных", marker = :square)  
plot!(p1, grouped.beta, grouped.mean_deaths ./ 3000, label = "Доля  
↪ умерших", marker = :diamond)  
  
savefig(plotsdir("beta_scan.png"))  
  
println("Результаты сохранены в data/beta_scan_all.csv и  
↪ plots/beta_scan.png")
```

p1

Результаты сохранены в data/beta_scan_all.csv и plots/beta_scan.png



1.7 Исследование эффекта миграции

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots, CSV, Random

include(sourcedir("sir_model.jl"))
```

```
└ Warning: DrWatson could not find find a project file by recursively checking
  | (given dir: /home/eavernikovskaya)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

total_count (generic function with 1 method)

Функция для создания матрицы миграции с заданной интенсивностью

```
function create_migration_matrix(C, intensity)
    M = ones(C, C) .* intensity ./ (C-1)
    for i = 1:C
        M[i, i] = 1 - intensity
    end
    return M
end
```

create_migration_matrix (generic function with 1 method)

Функция для измерения времени достижения пика

```
function peak_time(p)
```

```

migration_rates = create_migration_matrix(p[:C],
↳ p[:migration_intensity])
model = initialize_sir(;
    Ns = p[:Ns],
    β_und = p[:β_und],
    β_det = p[:β_det],
    infection_period = p[:infection_period],
    detection_time = p[:detection_time],
    death_rate = p[:death_rate],
    reinfection_probability = p[:reinfection_probability],
    Is = p[:Is],
    seed = p[:seed],
    migration_rates = migration_rates,
)
infected_frac(model) = count(a.status == :I for a in
↳ allagents(model)) / nagents(model)
peak = 0.0
peak_step = 0

for step = 1:p[:n_steps]
    agent_ids = collect(allids(model))
    for id in agent_ids
        agent = try
            model[id]

        catch
            nothing
        end
        if agent != nothing
            sir_agent_step!(agent, model)
        end
    end
end

```

```

        end
    end
    frac = infected_frac(model)
    if frac > peak
        peak = frac
        peak_step = step
    end
end
return (peak_time = peak_step, peak_value = peak)
end

```

peak_time (generic function with 1 method)

Сканирование интенсивности миграции

```

migration_intensities = 0.0:0.1:0.5
seeds = [42, 43, 44]

params_list = []
for mig in migration_intensities
    for s in seeds
        push!(
            params_list,
            Dict(
                :migration_intensity => mig,
                :C => 3,
                :Ns => [1000, 1000, 1000],
                :β_und => [0.5, 0.5, 0.5],
                :β_det => [0.05, 0.05, 0.05],
            )
        )
    end
end

```

```

      :infection_period ⇒ 14,
      :detection_time ⇒ 7,
      :death_rate ⇒ 0.02,
      :reinfection_probability ⇒ 0.1,
      :Is ⇒ [1, 0, 0],
      :seed ⇒ s,
      :n_steps ⇒ 150,
    ),
  )
end
end

```

Запуск

```

results = []
for params in params_list
  data = peak_time(params)
  push!(results, merge(params, Dict(pairs(data))))
  println("Завершён эксперимент с migration_intensity =
    ↳ $(params[:migration_intensity]), seed =
    ↳ $(params[:seed])",)
end

```

```

Завершён эксперимент с migration_intensity = 0.0, seed = 42
Завершён эксперимент с migration_intensity = 0.0, seed = 43
Завершён эксперимент с migration_intensity = 0.0, seed = 44
Завершён эксперимент с migration_intensity = 0.1, seed = 42
Завершён эксперимент с migration_intensity = 0.1, seed = 43
Завершён эксперимент с migration_intensity = 0.1, seed = 44

```

Завершён эксперимент с migration_intensity = 0.2, seed = 42
Завершён эксперимент с migration_intensity = 0.2, seed = 43
Завершён эксперимент с migration_intensity = 0.2, seed = 44
Завершён эксперимент с migration_intensity = 0.3, seed = 42
Завершён эксперимент с migration_intensity = 0.3, seed = 43
Завершён эксперимент с migration_intensity = 0.3, seed = 44
Завершён эксперимент с migration_intensity = 0.4, seed = 42
Завершён эксперимент с migration_intensity = 0.4, seed = 43
Завершён эксперимент с migration_intensity = 0.4, seed = 44
Завершён эксперимент с migration_intensity = 0.5, seed = 42
Завершён эксперимент с migration_intensity = 0.5, seed = 43
Завершён эксперимент с migration_intensity = 0.5, seed = 44

Сохраняем все прогоны

```
df = DataFrame(results)
CSV.write(datadir("migration_scan_all.csv"), df)
```

"/home/eavernikovskaya/work/study/2025-2026/Имитационное моделирование/simulation
modeling/labs/lab03/project/data/migration_scan_all.csv"

Усреднение по повторам

```
using Statistics
grouped = combine(
    groupby(df, [:migration_intensity]),
    :peak_time => mean => :mean_peak_time,
    :peak_value => mean => :mean_peak_value,
)
```

	migration_intensity	mean_peak_time	mean_peak_value
	Float64	Float64	Float64
1	0.0	15.0	0.333259
2	0.1	22.0	1.0
3	0.2	16.0	1.0
4	0.3	17.0	1.0
5	0.4	19.0	1.0
6	0.5	20.0	1.0

Визуализация

```

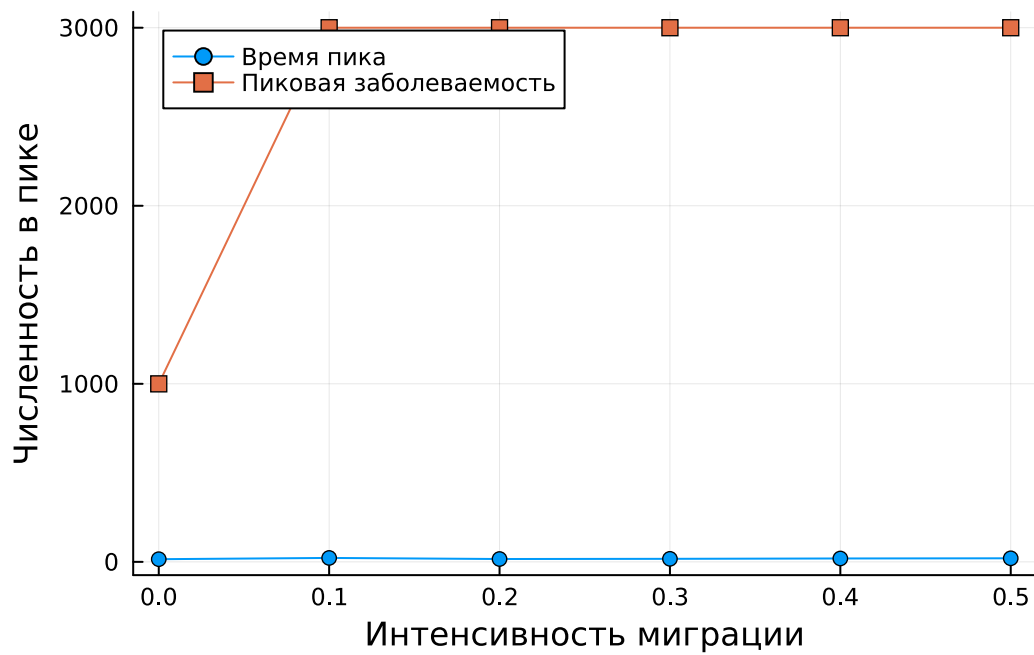
p2 = plot(
    grouped.migration_intensity,
    grouped.mean_peak_time,
    marker = :circle,
    xlabel = "Интенсивность миграции",
    ylabel = "Время до пика (дни)",
    label = "Время пика",
)
plot!(
    p2,
    grouped.migration_intensity,
    grouped.mean_peak_value .* 3000,
    marker = :square,
    xlabel = "Интенсивность миграции",
    ylabel = "Численность в пике",
    label = "Пиковая заболеваемость",
)
savefig(plotsdir("migration_effect.png"))

```

```
println("Результаты сохранены в data/migration_scan_all.csv и  
↳ plots/migration_effect.png")
```

p2

Результаты сохранены в data/migration_scan_all.csv и plots/migration_effect.png



1.8 Сводная визуализация результатов

```
using DrWatson
@quickactivate "project"
using Agents, DataFrames, Plots, CSV

include(sourcedir("sir_model.jl"))
```

```
└ Warning: DrWatson could not find find a project file by recursively checking
```

```
| (given dir: /home/eavernikovskaya)
└ @ DrWatson ~/.julia/packages/DrWatson/2QF5p/src/project_setup.jl:84
```

```
total_count (generic function with 1 method)
```

Загружаем результаты сканирования

```
df = CSV.read(datadir("beta_scan_all.csv"), DataFrame)
```

	n_steps	final_rec	Ns	death_rate	Is	beta	infection
	Int64	Float64	String31	Float64	String15	Float64	Int64
1	100	0.000333333	[1000,1000,1000]	0.02	[0,0,1]	0.1	14
2	100	0.00266667	[1000,1000,1000]	0.02	[0,0,1]	0.1	14
3	100	0.000333333	[1000,1000,1000]	0.02	[0,0,1]	0.1	14
4	100	0.0400976	[1000,1000,1000]	0.02	[0,0,1]	0.2	14
5	100	0.001	[1000,1000,1000]	0.02	[0,0,1]	0.2	14
6	100	0.000333333	[1000,1000,1000]	0.02	[0,0,1]	0.2	14
7	100	0.00748503	[1000,1000,1000]	0.02	[0,0,1]	0.3	14
8	100	0.0606509	[1000,1000,1000]	0.02	[0,0,1]	0.3	14
9	100	0.000333333	[1000,1000,1000]	0.02	[0,0,1]	0.3	14
10	100	0.080938	[1000,1000,1000]	0.02	[0,0,1]	0.4	14
11	100	0.0298954	[1000,1000,1000]	0.02	[0,0,1]	0.4	14
12	100	0.000666667	[1000,1000,1000]	0.02	[0,0,1]	0.4	14
13	100	0.00938086	[1000,1000,1000]	0.02	[0,0,1]	0.5	14
14	100	0.000370233	[1000,1000,1000]	0.02	[0,0,1]	0.5	14
15	100	0.000372995	[1000,1000,1000]	0.02	[0,0,1]	0.5	14
16	100	0.0142643	[1000,1000,1000]	0.02	[0,0,1]	0.6	14
17	100	0.00187829	[1000,1000,1000]	0.02	[0,0,1]	0.6	14
18	100	0.00380228	[1000,1000,1000]	0.02	[0,0,1]	0.6	14
19	100	0.0944487	[1000,1000,1000]	0.02	[0,0,1]	0.7	14
20	100	0.0026296	[1000,1000,1000]	0.02	[0,0,1]	0.7	14
21	100	0.0165414	[1000,1000,1000]	0.02	[0,0,1]	0.7	14
22	100	0.134969	[1000,1000,1000]	0.02	[0,0,1]	0.8	14
23	100	0.00224803	[1000,1000,1000]	0.02	[0,0,1]	0.8	14
24	100	0.0237378	[1000,1000,1000]	0.02	[0,0,1]	0.8	14
25	100	0.0473259	[1000,1000,1000]	0.02	[0,0,1]	0.9	14
26	100	0.0622391	[1000,1000,1000]	0.02	[0,0,1]	0.9	14
27	100	0.0	[1000,1000,1000]	0.02	[0,0,1]	0.9	14
28	100	0.0	[1000,1000,1000]	0.02	[0,0,1]	1.0	14
29	100	0.0	[1000,1000,1000]	0.02	[0,0,1]	1.0	14
30	100	0.0	[1000,1000,1000]	0.02	[0,0,1]	1.0	14

Создаём составной график

```
p1 = plot(df.beta, df.peak, label = "Пик", xlabel = "β", ylabel =  
    ↪ "Доля инфицированных")  
plot!(p1, df.beta, df.final_inf, label = "Конечная")  
  
p2 = plot(df.beta, df.deaths, xlabel = "β", ylabel = "Число  
    ↪ умерших")  
p3 = plot(df.beta, df.final_rec, xlabel = "β", ylabel = "Доля  
    ↪ выздоровевших")  
  
combined = plot(p1, p2, p3, layout = (3, 1), size = (800, 900))  
  
savefig(plotsdir("comprehensive_analysis.png"))  
  
combined
```

